TP 3 : Neo4j

idir.benouaret@epita.fr

1 Introduction et Contexte

Neo4j est une base de données orientée graphes spécialement conçue pour gérer les relations complexes entre les données. Dans le cadre du projet Tiny-X, nous l'utiliserons pour implémenter le service social, qui repose sur les interactions entre les utilisateurs et leurs contenus. Grâce à son modèle basé sur les nœuds et les relations, Neo4j nous permet de gérer efficacement des fonctionnalités comme le suivi (follow) d'un utilisateur, le blocage, l'identification des amis communs, ainsi que l'analyse des recommandations (suggestion de comptes à suivre ou de contenus pertinents). De plus, il facilite la modélisation des interactions avec les tweets, en permettant d'optimiser des requêtes complexes, comme retrouver les publications les plus likées dans un réseau proche ou identifier les influenceurs d'une communauté. Son approche relationnelle native offre ainsi une solution performante pour modéliser et exploiter les connexions entre utilisateurs.

Ce TP a pour objectif de vous familiariser avec Neo4j et Cypher, son langage d'interrogation. Vous allez dans la suite charger un jeu de données et écrire les requêtes Cypher permettant chacune de répondre à des besoins bien précis.

Remarque : Si vous êtes en salle machine de l'école : sauter l'étape d'installation et utiliser la commande suivante sur votre terminal pour lancer Neo4j

```
mkdir ~/neo4j
NEO4J_HOME=~/neo4j neo4j start
```

Les identifiants par défaut sont : `user neo4j password neo4j`

Il est aussi possible d'utiliser Docker :

```
docker run --publish=7474:7474 --publish=7687:7687 --env=NEO4J_AUTH=none neo4j
```

2 Importation du jeu de données

Télécharger les jeux de données à partir des "givens files"

Ensuite, dans la console Neo4j, exécutez successivement les instructions Cypher suivantes :

Pensez bien évidement à adapter les chemins d'accès aux fichiers : neo4j cherche les fichiers dans le dossier /var/lib/neo4j/import

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/users.
  csv' AS users
CREATE (u:User {id : toInteger(users.id), sex : users.sex, age : toInteger(users.age)
  , occupation : users.occupation, zip_code : users.zip_code})
```

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/movies.
    csv' AS movies
CREATE (m:Movie {id : toInteger(movies.id), title : movies.title, date :toInteger(
    movies.date)})
```

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/genres.
    csv' AS genres
CREATE (u:Genre {id : toInteger(genres.id), name : genres.name})
```

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/
    movies_genres.csv' AS movies_genres
MERGE (m : Movie { id : toInteger(movies_genres.mov_id) })
MERGE (g : Genre { id : toInteger(movies_genres.genre) })
CREATE (m)-[:CATEGORIZED_AS]->(g)
```

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/ratings
  .csv' AS ratings
MERGE (u : User { id : toInteger(ratings.user_id) })
MERGE (m : Movie { id : toInteger(ratings.mov_id) })
CREATE (u)-[:RATED { note : toInteger(ratings.rating), timestamp : toInteger(ratings.
  timestamp) } ]->(m)
```

```
LOAD CSV WITH HEADERS FROM 'file:///home/idir/Desktop/NOSQL/TP-Neo4j/datasets/friends.
    csv' AS friends
MERGE (u1:User { id : toInteger(friends.user1_id) })
MERGE (u2:User { id : toInteger(friends.user2_id) })
CREATE (u1)-[:FRIEND_OF]->(u2)
```

La base de données *MovieLens* comprend trois types de nœuds :

- **User** : représente un utilisateur de la base de données *MovieLens*. Un utilisateur est défini par les attributs suivants : id : int, sex : string, age : int, occupation : string, zip code : int
- **Movie** : représente un film de la base de données *MovieLens*. Un film est défini par les attributs suivants : id : int, title : string, date : int
- **Genre** : représente le genre d'un film et contient les attributs suivants : id : int, name : string

La base de données *MovieLens* exploite également trois types de relations :

- **RATED** : représente la note qu'a exprimé un utilisateur à propos d'un film qu'il a regardé. La relation contient les attributs suivants : note : int, timestamp : int
- **FRIEND_OF** : représente un lien d'amitié entre deux utilisateurs de la base de données. A noter que comme toute relation dans Neo4j, celle-ci est orientée. En particulier, dans notre cas, elle est aussi symétrique, i.e., A est ami avec B implique que B est ami avec A
- **CATEGORIZED_AS** : permet d'assigner un genre à un film.

Consignes pour la soumission de votre travail

Lisez attentivement les instructions suivantes avant de commencer les exercices

- Répondez à chaque requête dans un fichier `tp3/qN.cypher` ou N est le numéro de la question
- Par exemple, la réponse à la question 4 dans un fichier `tp3/q4.cypher`
- Pour exécuter et tester vos requêtes :

```
cypher-shell -f tp3/q1.cypher
```

- Les fichiers à soumettre sont : `tp3/q1.cypher, tp3/q2.cypher, ..., tp3/q19.cypher`
- Faire attention à bien nommer toutes vos colonnes sur les résultats à chaque fois que cela est indiqué dans la question

Exercice I : Requêtes

Q1 – Combien y a-t-il d'utilisateurs dans la base de données ?

- `https://neo4j.com/docs/cypher-manual/current/queries/basic/#find-nodes`
- `https://neo4j.com/docs/cypher-manual/current/functions/aggregating/#functions-count`

La colonne du résultat doit être nommée : `numberOfUsers`

Q2 – Quel est l'âge et le sexe de l'utilisateur qui a pour id la valeur 42 ?

- ◇ `https://neo4j.com/docs/cypher-manual/current/clauses/match/#match-with-labels`

Les colonnes du résultat doivent être nommées : `age, sex`

Q3 – Listez les femmes qui sont artistes (occupation='artist') ?

La colonne des nœuds doit être nommée : `womenArtists`

Q4 – Listez les femmes qui sont artistes ou auteurs (occupation='writer') ?

- ◇ `https://neo4j.com/docs/cypher-manual/current/clauses/where/#boolean-operations`

La colonne des nœuds doit être nommée : `womenArtistOrWriter`

Q5 – Quel est l'âge moyen des étudiants (occupation='student') de la base de données ?

- ◇ `https://neo4j.com/docs/cypher-manual/current/functions/aggregating/`

Les colonnes du résultat doivent être nommées : `averageAge`

Q6 – Quel est l'âge moyen des utilisateurs pour chacune des occupation ? Ordonner les réponses par ordre décroissant des moyennes

- ◇ `https://neo4j.com/docs/cypher-manual/current/clauses/order-by/`

Les colonnes du résultat doivent être nommées : `occupation, averageAge`

Q7 – Quel est le nombre d'utilisateurs pour chaque occupation ?

Les colonnes du résultat doivent être nommées : `occupation, count`

Q8 – Combien y a-t-il d'occupations pour lesquelles l'âge moyen des utilisateurs est strictement supérieur à 42 ans ?

- ◇ `https://neo4j.com/docs/cypher-manual/current/clauses/with/`

Les colonnes du résultat doivent être nommées : `numberOfOccupations`

Q9 – Listez les films sortis en 1994 ?

Remarque : Notez que la date de production d'un film n'est pas un attribut des nœuds Movie, mais apparaît entre parenthèses à la fin de chaque titre de film. Exploitez donc les expressions régulières pour répondre à la question

- ◇ `https://neo4j.com/docs/cypher-manual/current/expressions/predicates/string-operators/`

La colonne des nœuds doit être nommée `moviesIn1994`

Q10 – Au total, combien de notes contient la bases de données ? i.e., combien de relation de type *RATED*

- ◇ `https://neo4j.com/docs/cypher-manual/current/queries/concepts/#core-concepts-relationships`

Les colonnes du résultat doivent être nommées : `numberOfRatings`

Q11 – Quelles sont toutes les notes laissées par l'utilisateur qui a pour id la valeur 20 ? votre requête doit afficher le résultat sous forme de graphe reliant chaque note au film concerné.

Les colonnes doivent être nommées : `user, rating, movie`

Q12 – Calculez la moyenne des notes données par l'utilisateur qui a pour id la valeur 20

Les colonnes du résultat doivent être nommées : `averageRating`

Q13 – Combien d'utilisateurs ont noté le film intitulé "Jurassic Park (1993)" ? On suppose qu'un utilisateur peut noter un film au plus une seule fois.

Les colonnes du résultat doivent être nommées : `numberOfUsers`

Q14 – Quels sont les dix films qui ont reçu le plus de notes ? Afficher à chaque fois le titre du film et son nombre total de notes

Les colonnes du résultat doivent être nommées : `movieTitle, numberOfRatings`

Q15 – Combien de films ont reçu au moins une fois la note 1 ?

Les colonnes du résultat doivent être nommées : `numberOfMoviesWithRatingOne`

Q16 – Quels sont les films qui ont reçu au moins une fois la note 1, combien de fois ont-ils obtenu cette note, et qui sont les utilisateurs l'ayant attribué ?

Votre requête doit retourner une ligne par film (donc autant de lignes que la réponse à la question précédente) avec trois informations : le titre du film, le nombre de fois où il a reçu la note 1, et la liste des identifiants de chaque utilisateur qui lui a donné cette note.

Les colonnes du résultat doivent être nommées : `movieTitle, numberOfTimesRatedOne, userIds`. Les résultats doivent être triés par ordre ascendant selon le champ `movieTitle`. De plus, la liste des identifiants utilisateurs `userIds` doit être triée par ordre croissant.

- ◇ `https://neo4j.com/docs/cypher-manual/current/functions/aggregating/#functions-collect`

Q17 – Quels sont les films dont la note moyenne est strictement supérieure à 4 ? Ordonnez vos résultats par ordre décroissant de la note moyenne

Les colonnes du résultat doivent être nommées : `movieTitle, averageRating`

Q18 – Quel est l'utilisateur qui a le plus d'amis en commun avec l'utilisateur 33 ?

Les colonnes du résultat doivent être nommées : `userWithMostCommonFriends, commonFriendsCount`

Q19 – Quel est l'utilisateur qui a noté le plus grand nombre de films parmi ceux que l'utilisateur 42 a notés, et quel est ce nombre ?

Les colonnes du résultat doivent être nommées : `userId, numberOfMoviesRated`

A Installer et lancer Neo4j

Plusieurs possibilités pour installer Neo4j. Une des possibilités très simple est sans doute la version Desktop.

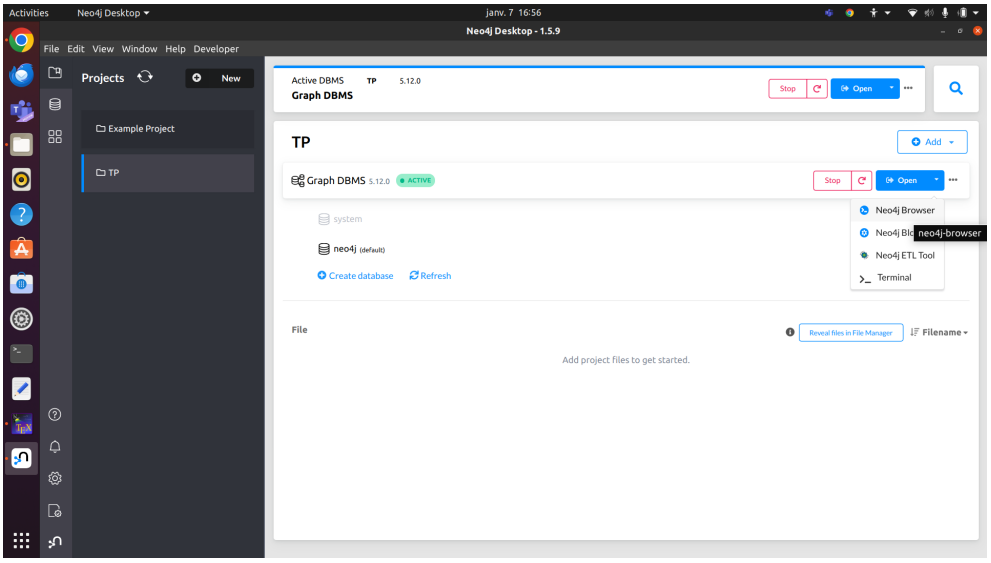

FIGURE 1 – Open Neo4j Browser

1. `https://neo4j.com/docs/operations-manual/current/installation/neo4j-desktop/`
2. Suivre les étapes indiquées, une image sera téléchargée. Il suffit ensuite de la lancer pour démarrer le serveur Neo4j.
3. Après avoir lancer Neo4j Desktop, vous pouvez créer une base de donnée pour réaliser le TP.

Pour permettre l'import de données depuis n'importe quel répertoire de votre machine, il faut commenter l'instruction `server.directories.import=import` dans le fichier de configuration. Pour cela, cliquer sur les 3 petits points à côté de votre base de données → settings → trouver la config et la commenter.

Ouvrez ensuite votre console Neo4j (voir figure 1), il est aussi possible d'ouvrir une console sur votre navigateur avec l'url `http://localhost:7474/`